

Sistemas Automatizados

Semana 2 — Processos, sensores e transdutores

Gabarito detalhado da preparação e da prática no Wokwi

Referência: slides 39–59 e 61–63 da apresentação da Semana 2.

UC - Sistemas Automatizados • Curso: Engenharia de Software • Prof. Calvetti

Finalidade: orientar a construção integral do protótipo pelo estudante, explicar as decisões de engenharia, apresentar resultados esperados e fornecer critérios objetivos para validar cada etapa.

Versão 1.0 • 20260814 - RC

1. Como utilizar este gabarito

Este documento acompanha a prática, mas não substitui a execução. O estudante deve criar o projeto vazio, montar o circuito, digitar o código, testar incrementalmente e registrar as evidências no próprio repositório GitHub.

Símbolo didático	Significado
O que fazer	A ação concreta no navegador, no circuito ou no código.
Por que fazer	A justificativa técnica ou pedagógica da ação.
Resultado esperado	O comportamento observável que confirma a etapa.
Se não funcionar	Uma sequência curta de diagnóstico.
Evidência	O registro que demonstra a execução e deve ser preservado.

Regra de ouro: alterar uma parte por vez. Quando muitas mudanças são feitas simultaneamente, torna-se difícil localizar a origem de um erro.

1.1 Escopo e limites da demonstração

- O botão representa uma entrada discreta; o potenciômetro, uma variável contínua; o DHT22, uma medição de temperatura; e o LED, uma saída de sinalização.
- A simulação comprova a coerência funcional da leitura, da decisão, da atuação e do diagnóstico.
- A simulação não comprova montagem industrial, imunidade a ruído, calibração, alcance, grau de proteção, compatibilidade eletromagnética ou segurança funcional.

1.2 Critério de conclusão

A atividade está concluída quando os quatro subsistemas funcionam, os estados são classificados com prioridade correta, os casos de teste são registrados e o repositório contém código, descrição, circuito e evidências.

2. Preparação operacional no Wokwi — slides 39 a 43

2.1 Criar o projeto vazio

Etapa 1 — acessar o ambiente

O que fazer: abra o navegador, acesse <https://wokwi.com> e selecione a criação de um novo projeto com Arduino Uno.

Por que fazer: o Arduino Uno oferece entradas digitais, entradas analógicas e bibliotecas suficientes para o protótipo, além de executar integralmente no navegador.

Resultado esperado: o editor apresenta o Arduino Uno, a área do circuito, o arquivo sketch.ino e o botão de execução.

Etapa 2 — adicionar os componentes

Na ferramenta de adição de componentes, inclua: um LED, um resistor de 220 Ω , um pushbutton, um potenciômetro e um DHT22. Os componentes são adicionados antes das ligações para que os pinos possam ser planejados e documentados.

Componente	Pino do Arduino	Ligação complementar	Razão
Pushbutton	D2	outro terminal em GND	entrada digital com pull-up interno
LED + resistor	D8	D8 $\rightarrow$ 220 Ω $\rightarrow$ ânodo; cátodo $\rightarrow$ GND	limitar corrente e sinalizar detecção
Potenciômetro	A0	VCC $\rightarrow$ 5 V; GND $\rightarrow$ GND; SIG $\rightarrow$ A0	produzir tensão variável
DHT22	D4	VCC $\rightarrow$ 5 V; GND $\rightarrow$ GND; DATA $\rightarrow$ D4	fornecer temperatura simulada

Atenção: no LED, o ânodo é o terminal positivo e o cátodo deve ir ao GND. O resistor pode ficar antes ou depois do LED no mesmo ramo; sua função elétrica permanece a limitação de corrente.

Etapa 3 — definir pinos antes de ligar

A definição prévia evita conexões contraditórias e transforma o circuito em uma interface documentada. Neste gabarito, BTN=2, LED=8, POT=A0 e DHTPIN=4. Se outros pinos forem escolhidos, circuito e código deverão ser alterados de forma consistente.

2.2 Entender a estrutura mínima do programa

Estrutura inicial

```
void setup() {
  // configuração executada uma vez
}

void loop() {
  // lógica repetida continuamente
}
```

O que está acontecendo: `setup()` é executada uma única vez após o início ou reinício; `loop()` é repetida enquanto o controlador estiver energizado.

Por que isso importa: um sistema automatizado precisa primeiro preparar suas interfaces e, depois, repetir o ciclo de adquirir entradas, decidir, atuar e diagnosticar.

Função	Responsabilidade neste protótipo
<code>setup()</code>	configurar pinos, iniciar comunicação serial e inicializar o DHT22
<code>loop()</code>	ler botão, potenciômetro e temperatura; decidir o estado; comandar LED; publicar diagnóstico

2.3 Entrada e saída digitais

Programa digital mínimo

```
const int BTN = 2;
const int LED = 8;

void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  bool ativo = digitalRead(BTN) == LOW;
  digitalWrite(LED, ativo ? HIGH : LOW);
}
```

Explicação linha a linha

Trecho	O que faz	Por que é necessário
<code>const int BTN = 2</code>	associa o nome BTN ao pino D2	evita números sem significado espalhados pelo código
<code>INPUT_PULLUP</code>	ativa resistor interno para manter a entrada em HIGH	evita entrada flutuante e dispensa resistor externo
<code>digitalRead(BTN) == LOW</code>	interpreta LOW como botão pressionado	o botão conecta D2 ao GND quando acionado
<code>digitalWrite(...)</code>	comanda HIGH ou LOW no LED	transforma a decisão lógica em atuação visível

Conceito central: com `INPUT_PULLUP`, a lógica física fica invertida: liberado = HIGH e pressionado = LOW. A variável `ativo` corrige semanticamente essa inversão.

2.4 Leitura analógica introdutória

Programa analógico mínimo

```
const int POT = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int bruto = analogRead(POT);
  Serial.println(bruto);
  delay(200);
}
```

`analogRead(A0)` converte a tensão do cursor do potenciômetro em um número inteiro. No Arduino Uno, a resolução usual é de 10 bits: $2^{10} = 1024$ combinações, representadas de 0 a 1023.

`Serial.begin(9600)` prepara o canal de diagnóstico. `Serial.println(bruto)` envia uma amostra por linha ao Monitor Serial. O atraso de 200 ms limita a taxa de mensagens e facilita a leitura.

Posição aproximada	Valor bruto esperado	Interpretação
mínimo	próximo de 0	tensão próxima de 0 V
meio	próximo de 512	aproximadamente metade da escala
máximo	próximo de 1023	tensão próxima de 5 V

Verificação: gire o potenciômetro lentamente. O valor deve variar de modo contínuo e monotônico. Pequenas diferenças em relação a 0, 512 e 1023 não invalidam o teste.

3. Simulação funcional de sensores — slides 44 a 59

3.1 Enunciado resolvido

Construir uma estação de monitoramento com uma entrada digital, uma entrada analógica, uma temperatura simulada e uma sinalização. O botão deve controlar o LED; o potenciômetro e o DHT22 devem alimentar uma decisão com estados NORMAL, ATENÇÃO, ALARME e FALHA DE SENSOR; pelo menos seis casos devem ser testados.

3.2 Estratégia de construção incremental

Incremento	Entrada/saída adicionada	Critério para avançar
1	botão + LED	dois estados digitais confirmados
2	potenciômetro	mínimo, meio e máximo observados
3	classificação analógica	fronteiras 399/400 e 749/750 verificadas
4	DHT22	temperaturas válidas e falha tratada
5	decisão integrada	prioridades e sete testes confirmados

Por que incremental? cada incremento cria uma linha de base funcional. Se a etapa seguinte falhar, o estudante sabe qual mudança introduziu o defeito.

3.3 Montagem do botão e do LED

Ligue um lado do pushbutton ao D2 e o lado eletricamente oposto ao GND. Ligue D8 ao resistor de $220\ \Omega$, o resistor ao ânodo do LED e o cátodo ao GND.

Depois, execute o programa digital mínimo. Botão liberado deve manter o LED apagado; botão pressionado deve acendê-lo.

Caso	Entrada observada	Saída esperada	Conclusão do gabarito
D1	liberado / HIGH	LED apagado	aprovado se não houver acionamento espontâneo
D2	pressionado / LOW	LED aceso	aprovado se a resposta for imediata
D3	cinco acionamentos	cinco respostas	aprovado se não houver perda de evento visível
D4	mantido pressionado	LED permanece aceso	aprovado enquanto LOW permanecer ativo

3.4 Acrescentar e classificar o potenciômetro

Conecte VCC a 5 V, GND a GND e SIG a A0. Leia o valor bruto e aplique os limites especificados no enunciado.

Classificação do sinal analógico

```
int bruto = analogRead(A0);

if (bruto < 400) {
  Serial.println("NORMAL");
} else if (bruto < 750) {
  Serial.println("ATENCAO");
} else {
  Serial.println("ALARME");
}
```


A primeira condição cobre 0 a 399. Quando ela é falsa, sabe-se que bruto é pelo menos 400; por isso, o segundo teste bruto < 750 cobre 400 a 749. O else restante cobre 750 a 1023.

Valor	Condição satisfeita	Resultado correto	Por que testar
399	bruto < 400	NORMAL	último valor da primeira faixa
400	bruto < 750	ATENÇÃO	primeiro valor da segunda faixa
749	bruto < 750	ATENÇÃO	último valor da segunda faixa
750	else	ALARME	primeiro valor da terceira faixa

Erro comum: usar ≤ 400 no primeiro teste desloca a fronteira e contradiz o requisito 0–399. Os testes de fronteira revelam esse tipo de erro.

3.5 Acrescentar o DHT22

Etapa 1 — circuito

Adicione o DHT22. Ligue VCC a 5 V, GND ao GND e DATA ao D4. A alimentação comum é necessária para que Arduino e sensor compartilhem a mesma referência elétrica.

Etapa 2 — biblioteca

No gerenciador de bibliotecas do projeto, adicione “DHT sensor library” (Adafruit). Se o ambiente solicitar a dependência “Adafruit Unified Sensor”, aceite sua inclusão. A biblioteca encapsula o protocolo de comunicação do DHT22 e fornece `readTemperature()`.

Etapa 3 — programa mínimo

Leitura segura da temperatura

```
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float t = dht.readTemperature();
  if (isnan(t)) {
    Serial.println("FALHA DE SENSOR");
  } else {
    Serial.println(t);
  }
  delay(1000);
}
```

`readTemperature()` retorna um número em graus Celsius quando a leitura é válida. Se falhar, pode retornar NaN (“not a number”). O teste `isnan(t)` impede que uma falha de instrumento seja interpretada como processo normal.

Resultado esperado: ao alterar a temperatura do DHT22 no simulador, o Monitor Serial deve apresentar o novo valor. Uma leitura inválida deve produzir FALHA DE SENSOR.

3.6 Código integrado completo — solução de referência

```
#include <DHT.h>

const int BTN = 2;
const int LED = 8;
const int POT = A0;
#define DHTPIN 4
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  bool detectado = digitalRead(BTN) == LOW;
  digitalWrite(LED, detectado ? HIGH : LOW);

  int bruto = analogRead(POT);
  float t = dht.readTemperature();

  Serial.print("BTN=");
  Serial.print(detectado ? "PRESSIONADO" : "LIBERADO");
  Serial.print(" | POT=");
  Serial.print(bruto);
  Serial.print(" | TEMP=");

  if (isnan(t)) {
    Serial.print("INVALIDA | ESTADO=FALHA DE SENSOR");
  } else {
    Serial.print(t, 1);
    Serial.print(" C | ESTADO=");
    if (t >= 40.0 || bruto >= 750) {
      Serial.print("ALARME");
    } else if (t >= 30.0 || bruto >= 400) {
      Serial.print("ATENCAO");
    } else {
      Serial.print("NORMAL");
    }
  }

  Serial.println();
  delay(1000);
}
```

Por que a ordem das decisões é importante?

A falha é avaliada antes das faixas de processo porque uma medida inválida não sustenta uma decisão de normalidade. Em seguida vem ALARME, depois ATENÇÃO. NORMAL é o caso residual: só ocorre se ambas as variáveis forem válidas e estiverem abaixo dos limites de atenção.

Prioridade	Condição	Saída
1	temperatura inválida	FALHA DE SENSOR
2	$t \geq 40\text{ °C}$ e bruto ≥ 750	ALARME
3	$t \geq 30\text{ °C}$ ou bruto ≥ 400	ATENÇÃO
4	demais combinações válidas	NORMAL

3.7 Execução e registro dos casos de teste integrados

Para cada caso, ajuste primeiro o potenciômetro e a temperatura; em seguida, posicione o botão; aguarde uma nova linha no Monitor Serial; compare a resposta obtida com a esperada; registre a evidência.

Caso	Pot.	Temp.	Botão	Resposta esperada	Justificativa
T1	200	25 °C	livre	NORMAL; LED off	todas as entradas em faixa normal
T2	500	25 °C	livre	ATENÇÃO; LED off	potenciômetro entre 400 e 749
T3	800	25 °C	livre	ALARME; LED off	potenciômetro ≥ 750
T4	200	35 °C	livre	ATENÇÃO; LED off	temperatura entre 30 e $<40\text{ °C}$
T5	200	45 °C	livre	ALARME; LED off	temperatura $\geq 40\text{ °C}$
T6	200	25 °C	pressionado	NORMAL; LED on	botão atua no LED sem alterar o estado
T7	—	inválida	livre	FALHA DE SENSOR	diagnóstico tem a maior prioridade

Como produzir evidências úteis

- Capture o circuito completo com os rótulos e ligações visíveis.
- Capture o Monitor Serial mostrando casos representativos, especialmente as fronteiras e a falha.
- Registre em uma tabela: entrada aplicada, saída esperada, saída obtida e situação aprovado/reprovado.
- Se houver divergência, corrija o projeto e repita o caso; não apague o raciocínio do diagnóstico do relatório.

3.8 Diagnóstico de problemas frequentes

Sintoma	Causa provável	Verificação e correção
LED nunca acende	polaridade ou pino incorreto	confirme D8, resistor em série, ânodo/cátodo e GND
LED fica sempre aceso	botão ligado incorretamente ou lógica invertida	confirme D2–GND e interprete pressionado como LOW
entrada digital oscila	entrada flutuante	mantenha pinMode(BTN, INPUT_PULLUP)
potenciômetro não varia	terminal central fora de A0	confirme SIG→A0 e extremos→5V/GND
Monitor Serial vazio	serial não iniciada ou simulação parada	confirme Serial.begin(9600), println e execução
DHT não compila	biblioteca ausente ou include divergente	adicione DHT sensor library e use #include <DHT.h>
temperatura inválida	DATA/pino/tipo incorretos	confirme DATA→D4, DHTPIN 4 e DHTTYPE DHT22
estado incorreto na fronteira	operador relacional errado	teste 399/400, 749/750 e 29,9/30/39,9/40 °C

3.9 O que a simulação demonstra — e o que não demonstra

Demonstra funcionalmente	Não valida fisicamente
aquisição de entradas digitais e analógicas	alcance, tempo de resposta real e tolerâncias
prioridade e coerência da lógica	ruído elétrico, aterramento e compatibilidade eletromagnética
atuação e diagnóstico no Monitor Serial	montagem, cabeamento, invólucro e grau de proteção
testes de fronteira e rastreabilidade	calibração metrológica e segurança funcional

4. Exercícios adicionais — slides 61 a 63

4.1 Exercício adicional 1 — prioridade de alarmes

Objetivo: ampliar e tornar explícita a prioridade entre falha de instrumento, alarme, atenção e normalidade.

Passo 1 — definir a regra antes do código

Ordem	Estado	Regra
1	FALHA	isnan(t)
2	ALARME	$t \geq 40\text{ °C}$ ou $\text{bruto} \geq 750$
3	ATENÇÃO	$t \geq 30\text{ °C}$ ou $\text{bruto} \geq 400$
4	NORMAL	nenhuma regra anterior satisfeita

Passo 2 — encapsular a classificação

```
const char* classificar(float t, int bruto) {  
    if (isnan(t)) return "FALHA DE SENSOR";  
    if (t >= 40.0 || bruto >= 750) return "ALARME";  
    if (t >= 30.0 || bruto >= 400) return "ATENCAO";  
    return "NORMAL";  
}
```

A função recebe as medidas e devolve o estado. Separar a decisão facilita testes, reduz repetição e permite reutilizar a mesma regra em outras saídas.

Passo 3 — cinco novos casos de teste resolvidos

Caso	Pot.	Temp.	Resultado	Justificativa
P1	900	45 °C	ALARME	duas condições de alarme; saída continua única
P2	500	45 °C	ALARME	alarme de temperatura supera atenção do potenciômetro
P3	900	35 °C	ALARME	alarme do potenciômetro supera atenção térmica
P4	500	35 °C	ATENÇÃO	duas condições de atenção, nenhuma de alarme
P5	900	inválida	FALHA DE SENSOR	falha de instrumento supera o alarme do potenciômetro

Resposta conceitual: P5 é o teste decisivo. Se o programa retornar ALARME, a falha foi avaliada tarde demais e existe risco de confiar em uma medição inválida.

4.2 Exercício adicional 2 — histerese

Problema: sem histerese, pequenas oscilações ao redor de 40 °C alternam repetidamente entre ATENÇÃO e ALARME. A histerese cria dois limites: um para ligar e outro, inferior, para desligar.

Passo 1 — guardar o estado anterior

Crie uma variável booleana fora de loop(). Ela precisa persistir entre ciclos; se fosse declarada dentro da função, o histórico seria perdido a cada repetição.

```
bool alarmeTemperatura = false;
```


Passo 2 — aplicar limites diferentes

```
void atualizarAlarmeTemperatura(float t) {
    if (isnan(t)) return;

    if (!alarmeTemperatura && t >= 40.0) {
        alarmeTemperatura = true;    // liga em 40 °C
    } else if (alarmeTemperatura && t <= 37.0) {
        alarmeTemperatura = false;   // desliga em 37 °C
    }
}
```

Quando o alarme está desligado, apenas $t \geq 40$ o ativa. Quando está ligado, temperaturas entre > 37 e < 40 mantêm o estado. Apenas $t \leq 37$ o desativa.

Passo 3 — integrar com a classificação

```
const char* classificarComHisterese(float t, int bruto) {
    if (isnan(t)) return "FALHA DE SENSOR";

    atualizarAlarmeTemperatura(t);

    if (alarmeTemperatura || bruto >= 750) return "ALARME";
    if (t >= 30.0 || bruto >= 400) return "ATENCAO";
    return "NORMAL";
}
```

Passo 4 — sequência de subida e descida resolvida

Considere bruto=200 durante toda a sequência, de modo que apenas a temperatura determine o estado.

Amostra	Temperatura	Estado anterior do alarme	Ação	Saída
H1	36 °C	desligado	não atinge 40	ATENÇÃO
H2	39 °C	desligado	ainda não atinge 40	ATENÇÃO
H3	40 °C	desligado	liga o alarme	ALARME
H4	39 °C	ligado	mantém: ainda >37	ALARME
H5	38 °C	ligado	mantém: ainda >37	ALARME
H6	37 °C	ligado	desliga o alarme	ATENÇÃO
H7	36 °C	desligado	permanece desligado	ATENÇÃO
H8	29 °C	desligado	abaixo do limite de atenção	NORMAL

Por que H6 resulta em ATENÇÃO? a histerese desliga o alarme em 37 °C, mas a regra de atenção continua válida para temperaturas a partir de 30 °C.

Teste complementar recomendado

Repita a sequência 39,8 → 40,0 → 39,9 → 39,7 → 37,1 → 37,0. O sistema deve ligar em 40,0, permanecer ligado até 37,1 e desligar somente em 37,0.

4.3 Exercício adicional 3 — seleção industrial

Resposta-modelo: monitoramento de uma câmara térmica de processo. O sistema precisa detectar porta fechada, pressão da linha de circulação e temperatura interna. As três entradas didáticas do protótipo serão substituídas por instrumentos industriais coerentes.

Entrada didática	Mensurando/processo	Tecnologia industrial selecionada
Pushbutton	presença/posição da porta	sensor indutivo ou chave fim de curso industrial
Potenciômetro	pressão da linha de circulação	transmissor de pressão 0–10 bar, saída 4–20 mA
DHT22	temperatura interna da câmara	RTD Pt100 + transmissor 4–20 mA

Justificativa detalhada

Entrada	Faixa	Interface	Ambiente e justificativa
Porta	dois estados	24 Vcc discreta	fim de curso robusto e simples; indutivo evita contato se houver alvo metálico
Pressão	0–10 bar	4–20 mA	sinal robusto para distância e ruído; 4 mA permite representar zero vivo
Temperatura	0–150 °C	Pt100 + 4–20 mA	Pt100 é estável e adequado à faixa; transmissor padroniza o sinal

Falhas detectáveis e não detectáveis

Instrumento	Exemplo detectável	Exemplo não detectável apenas pela entrada
sensor da porta	circuito aberto, se houver diagnóstico elétrico apropriado	sensor mecanicamente deslocado, mas ainda comutando
transmissor de pressão	corrente fora da faixa diagnóstica, por exemplo <3,6 mA	deriva dentro da faixa que ainda parece plausível
Pt100 + transmissor	ruptura do elemento indicada pelo transmissor	erro sistemático pequeno sem comparação/calibração

O que o Wokwi não valida nessa seleção

O protótipo não valida faixa real, exatidão, repetibilidade, tempo de resposta, compatibilidade de materiais, grau IP, temperatura ambiente, vibração, ruído, segurança elétrica, instalação, calibração, SIL/PL ou conformidade normativa. Ele valida somente a estrutura funcional e documental da decisão.

Conclusão do exercício: selecionar um sensor não é apenas escolher “algo que mede”. É compatibilizar mensurando, faixa, interface, ambiente, desempenho metrológico, diagnóstico e risco do processo.

5. Entrega no GitHub — estrutura mínima recomendada

A entrega deve permitir que outra pessoa compreenda o problema, reproduza a lógica e verifique as evidências sem depender de explicações orais.

Organização sugerida do repositório

```
semana-02-sensores/  
├── README.md  
├── sketch.ino  
├── diagram.json  
├── libraries.txt  
├── evidencias/  
│   ├── circuito.png  
│   ├── testes-digitais.png  
│   ├── testes-integrados.png  
│   └── histerese.png  
└── testes/  
    └── casos-de-teste.md
```

5.1 Conteúdo do README.md

- objetivo do protótipo e relação entre componentes didáticos e funções industriais;
- mapa de pinos e breve descrição do circuito;
- regras de NORMAL, ATENÇÃO, ALARME e FALHA;
- procedimento de execução no Wokwi;
- tabela de casos com esperado, obtido e situação;
- limitações da simulação;
- identificação do autor e data da execução.

5.2 Checklist final do estudante

Verificação	Critério de aceite
Circuito	BTN D2, LED D8 com resistor, POT A0 e DHT22 D4 ligados corretamente
Código	compila e executa sem erro
Digital	LOW/HIGH e LED respondem como especificado
Analógico	valores e fronteiras 399/400/749/750 verificados
Temperatura	faixas e leitura inválida tratadas
Prioridade	falha > alarme > atenção > normal
Histerese	liga em 40 °C e desliga em ≤ 37 °C
Testes	casos obrigatórios e adicionais registrados
GitHub	arquivos, descrição e evidências presentes

6. Síntese conceitual

A prática demonstra a cadeia de um sistema automatizado: o processo produz grandezas; sensores e transdutores as convertem em sinais; o controlador adquire esses sinais; regras de decisão classificam o estado; uma saída atua ou sinaliza; e o diagnóstico informa condições anormais.

Conceito	Manifestação no protótipo
Entrada discreta	botão: HIGH/LOW
Entrada analógica	potenciômetro: 0–1023
Transdução	DHT22 converte temperatura em informação digital
Atuação	LED responde ao botão
Decisão	limiares classificam normal, atenção e alarme
Diagnóstico	NaN gera falha de sensor
Estado com memória	histerese depende do estado anterior
Validação	casos de teste e evidências demonstram comportamento

Aprendizagem esperada: ao concluir, o estudante deve ser capaz de montar uma simulação funcional, explicar cada decisão, testar fronteiras, tratar falhas e reconhecer os limites entre uma prova funcional e uma validação industrial.

Referências de apoio

Apresentação da UC Sistemas Automatizados — Semana 2: Processos, sensores e transdutores, slides 39–59 e 61–63.

Arduino. Language Reference: digitalRead, digitalWrite, analogRead, pinMode e Serial.

Wokwi. Documentação do simulador Arduino e do componente DHT22.

Adafruit. DHT sensor library — interface de leitura dos sensores DHT.